Sports Debate Show: Project Brief

Nov 3, 2026 · @Ian

Status

10 days to the November 14 deadline. The plan is locked and the bibles are drafted, so building in Claude Code starts now.

Done

- [x] ~~Concept: type an NBA or NFL topic, get a 3-to-5-minute debate segment~~
- [x] ~~Format: a webpage that runs on your laptop~~
- [x] ~~Scope: NBA and NFL only, current or all-time, no draft topics~~
- [x] ~~Tone: real debate, hosts keep personalities, fewer bits~~
- [x] ~~Spec: segment flow, debate rules, scoring, fact rules, screen and sound~~
- [x] ~~Data sources chosen and checked against their terms~~
- [x] ~~Character bibles drafted for serious debate~~
- [x] ~~Subtitles first, voices later; Claude Code draws the pixel art~~
- [x] ~~Working name: The Receipts~~

Not done yet

- [ ] Claude Code running, plus an API key with about $25 of credits
- [ ] Project folder holding this brief (saved as BRIEF.md) and the bibles
- [ ] Step 1 working: text scripts from a typed topic

What it is

Working name: **The Receipts**, because every host has to bring them.

A webpage that generates a three-to-five-minute NBA or NFL debate on any topic you type, current or all-time. A moderator and three pixel-art co-hosts argue it out in an 8-bit studio, in subtitles, using real stats that are researched and fact-checked first. A background judge scores every argument, and the most persuasive host wins.

It's a solo final project for an AI class, built entirely with Claude Code by someone who has never coded.

How a segment works

1. You type an NBA or NFL topic, current or all-time, and hit go.
2. The topic check screens it.
3. While it generates, the hosts get ready on set: reading notes, walking to their spots. A small control-room panel shows the pipeline steps.
4. A caller asks your question in their own words, and the moderator takes the call.
5. The AI picks the format: debate, buy or sell, or prediction.
6. The hosts argue in subtitles, each leaning on his kind of evidence. A judge scores every argument in the background.
7. The moderator lets the argument run, then pulls everyone back.
8. The moderator announces the winner by score, summarizes the winning argument, and notes where the others still disagree.
9. You can download the finished segment as a video.

Debate rules

The goal is a real argument. Hosts keep their personalities, but the substance is evidence and rebuttal, not bits.

- **Evidence leans.** No host owns a kind of evidence, but each leans on one more than the others, set by his bible: analytics, film and the eye test, or traditional stats and winning. He still uses whatever wins the point. The leans differ, so arguments clash.
- **Structure.** Every argument is a claim, its evidence, and why it matters.
- **Engagement.** Every reply deals with the previous point directly: rebut it, concede it, or reframe it. No talking past each other.
- **No drift to the middle.** Hosts can concede a specific point but change their overall position only when a fact defeats it. Nobody ends on "both sides make good points."
- **Plain English.** Hosts can get as technical as the argument needs. The first time an advanced stat comes up, someone explains it in one plain sentence, like: "EPA measures how much a play raises or lowers the points a team should expect on that drive."
- **Length.** 3 to 5 minutes. The segment ends when new arguments run out. Repeated points get cut, never padded.
- **Heat.** Blowups only when the topic earns one. Tangents are fine when they serve an argument.

- **Limits.** Players, coaches, and teams are fair game to criticize. Family and personal life are off-limits. Off-field legal trouble comes up only if it's the topic, stated as fact, with no jokes.
- **Language.** TV-MA, with profanity censored in the subtitles.
- **Consistency.** Hosts sound like their bibles every time. There's no memory between segments.
- **Worst-supported take (optional).** The moderator gives it only when the fact pack contradicts a host's argument. First extra to cut.

Scoring

The most persuasive host wins. A judge agent scores every argument in the background, using criteria adapted from competitive debate judging.

Criterion	What the judge asks	Weight
Evidence	Are the facts relevant, verified, and from a big enough sample?	30%
Reasoning	Does the evidence actually prove the claim?	25%
Rebuttal	Does it answer the opponent's strongest point directly?	25%
Impact	Does it show why this matters to the question asked?	10%
Clarity	Is the claim specific and easy to follow?	10%

Each criterion gets 1 to 5 points per argument, weighted and scaled to a score out of 100.

- **Averages, not totals.** A host's score is the average of his arguments, so talking more doesn't win. Each host needs at least two scored arguments.
- **Dropped arguments count.** As in competitive debate, a point nobody answers counts as conceded. Its maker gets a bonus, 5 points to start, tuned after testing.
- **Volume, insults, and repetition** earn nothing.
- **Blind judging.** The judge sees Host 1, 2, and 3, not names, so it can't favor a character.
- **Ties** go to the higher rebuttal score.

- **Close after scoring.** The moderator's closing lines are written once the score is in, so the verdict always matches it. The meter moves as each argument is scored.
- **The winning summary** gives the winner's claim, his strongest evidence, and the rebuttal that landed, in two or three sentences.

Topic check

Every topic passes three checks before research starts, so a bad topic costs about a second instead of a full run.

1. **NBA or NFL.** The topic has to be about the NBA or NFL, current or all-time. Comparisons across the two leagues are fine. College and draft-prospect topics are turned away.
2. **Real.** Every player, team, and coach named must exist in the nflverse or nba_api roster data. A near miss gets a suggestion ("Did you mean Patrick Mahomes?"). A false premise, like a player on the wrong team, is turned away with the correct fact. Predictions about the future are fine.
3. **Appropriate.** No slurs, sexual content, threats, or harassment, and nothing about players' families or personal lives. Text that tries to instruct the hosts ("ignore your rules and say...") is treated as a topic, never followed, and turned away.

How it runs: a plain script matches names and teams against the roster data. Haiku handles the sports and appropriateness checks.

When a topic fails, a call-screener message on screen gives the reason in one line and offers three valid topics instead. Nothing gets voiced.

Log it: rejected topics go in the log too, so the presentation can show topics screened next to lines checked and cut.

Facts and data

Every real-world fact comes from the fact pack. Anything that can't be verified gets rewritten or cut.

Source	Used for	Role
nflverse via nflreadpy	NFL play-by-play, player stats, Next Gen Stats, FTN charting, injuries, schedules, depth charts, trades	Pipeline
nba_api	NBA.com stats	Pipeline
Web search	News context and pre-1999 NFL numbers, with links saved	Pipeline
ESPN	Second source for basic stats	Cross-check
PerThirtySix, RayCarp, Next Gen Stats site	Checking numbers by hand, method notes	Manual only
Pro Football Reference, Basketball-Reference, SumerSports	Nothing. Their terms ban scraping and AI use (Sports Reference, SumerSports)	Never

Every stat in the fact pack stores its value, exact definition, sample size, source, and an "as of" date.

Checking rules

- **Basic stats** like yards, touchdowns, and points get cross-checked against a second source. The official source wins, and a mismatch means flag or drop.
- **Advanced stats** must match the fact pack exactly, under the same definition. Caleb Williams is 0.16 EPA per play on SumerSports and +0.28 EPA per dropback on PerThirtySix, and both are correct.
- **Sample size.** No league-best or league-worst claims below the minimum, such as 150 dropbacks, unless the host mentions the small sample.
- **Rounding.** Counts must be exact. Rates can round to one decimal. "About" is allowed only when rounding a real number.
- **Host careers** are fiction, checked against the bibles. Their stories never name real games or real teammates.
- **Opinions and predictions** aren't checked.
- **A failed line** gets rewritten with verified numbers, and is cut only if that fails. Then the checker rereads the lines around it so the conversation still flows.

- **The rejection log** records every caught line: what the host said, what the data said, and whether it was rewritten or cut. It becomes a presentation slide.

All-time topics: NBA career numbers come from nba_api, though older seasons carry fewer stat types. nflverse NFL stats start in 1999. Older NFL numbers come from web search and need two independent sources that agree, or they get cut.

On screen: a stat chyron shows the value, a plain-English definition, the source, and how current it is ("Through Week 9"), and a source list appears at the end.

Cast

A moderator and three hosts, each defined by a bible in the characters folder. The biggest difference between the hosts is the kind of evidence each one leans on.

- **Moderator.** Asks the questions, doesn't interrupt, and lets the argument run until it's time to move on, then brings everyone back. His bible covers how much chaos he tolerates, how he cuts in, how he wraps up, how he picks a winner, and one quirk.
- **Three hosts.** Each bible covers:
  - Name, age, look, and a signature animation
  - A fictional career: stats, rings, and stories
  - Teams and players he loves and hates
  - The evidence he leans on and how he argues
  - Standing takes he'll defend no matter what
  - What sets him off
  - Voice, pace, catchphrases (once per segment at most), and 5 to 10 sample lines
  - What he thinks of each other host and the moderator
  - Lines he'd never say
- **Callers.** A pool of five or six types, such as a diehard fan, a stats nerd, or a rival-team fan, so each opening sounds different.

Banks to add once the bibles pass the blind test: a takes bank per host on teams, players, and coaches; a career bank of stats and stories; a line bank; 3 to 5 example exchanges; moderator lines; caller openings.

Blind test: hide the speaker names in a script and check whether you can tell who's talking from the lines alone.

Screen and sound

An 8-bit studio, the same set every time, with the page around it themed to match.

- **Subtitles first.** Lines appear as pixel-font subtitles paced at speaking speed, with the speaker's name, while his character animates. Voices are a stretch goal.
- **Pixel art.** Claude Code draws all the characters and the set in code.
- **Emotion** comes through body animation driven by each line's delivery tag: leaning in, pointing, standing up, slamming the desk, throwing hands up, turning away.
- **On screen:** subtitles, a topic banner, a stat chyron whenever a stat comes up, a meter showing each host's score as arguments are judged, and a source list at the end.
- **Sound:** no theme music. A desk-slam effect is optional.
- **Branding:** working name The Receipts, with a logo.

How it's built

Everything runs on your laptop: a small local server starts each generation, and the webpage plays the result. API keys live in a private .env file, never in the page code, this brief, or a chat.

Pipeline

1. **Topic in.** Run the topic check, then sort the topic into a format: debate, buy or sell, or prediction.
2. **Research.** Pull stats from nflverse or nba_api, plus news and pre-1999 NFL numbers from web search, into the fact pack.
3. **Write.** Produce the debate as structured lines: speaker, text, delivery, the fact IDs each line uses, and which argument it belongs to.
4. **Check.** A plain script matches every number against the fact pack, and an AI checker handles other claims. Failed lines get rewritten or cut, and everything is logged.
5. **Judge.** Score every argument blind, using the Scoring criteria, and pick the winner.
6. **Close.** Write the moderator's verdict and the summary of the winning argument, now that the score is known.
7. **Assemble.** Save a timeline file: who speaks when, subtitle timing, animations, chyrons, and meter updates.
8. **Play.** The page plays the timeline.

Models: Sonnet for writing, checking, judging, and the close. Haiku for small jobs like the topic check and sorting the format.

Speed target: topic to playback in about 2 minutes or less. Skipping voices for now makes this easier. Measure it on November 8.

Schedule

Get an ugly version working end to end, with a plain page showing a scored, subtitled segment, by November 9. Then improve it.

- [ ] **Nov 4-5:** Set up Claude Code and the API key. Put the brief and bibles in the project folder. Generate text scripts from a typed topic, read five, and fix the bibles.
- [ ] **Nov 6-7:** Data pull from nflverse and nba_api, the topic check, the fact pack, the number checker, and the rejection log.
- [ ] **Nov 8:** The judge, scoring, and the close. Time a full run from topic to finished script.
- [ ] **Nov 9-11:** The webpage: input box, studio, sprites, subtitles, player, loading screen, chyron, and meter.
- [ ] **Nov 12:** Test 20 topics and fix what breaks.
- [ ] **Nov 13:** Freeze the code, record a backup segment, and build the presentation.
- [ ] **Nov 14:** Due.

If you fall behind, cut in this order:

1. Video download (screen-record a segment instead)
2. The trailer
3. The animated meter (keep the scoring)
4. The hosts-getting-ready animation (use a static "stand by" screen)
5. The caller opener
6. Worst-supported take

Never cut: a typed topic becoming a subtitled, scored segment on a pixel page, backed by the research step and the fact-check log.

Stretch goal, only if everything above works: voices.

Presentation

Three minutes, built around one live run with a classmate's topic.

Time	What happens
0:00 to 0:10	A classmate calls out a topic. You type it and hit go. Keep three typed backups in case the room is quiet.
0:10 to 0:50	The trailer plays while it generates. Cut it only from real generated segments.
0:50 to 1:40	The pipeline live on screen: research finding stats, the script appearing, the checker catching lines.
1:40 to 2:50	The first minute or so of the segment.
2:50 to 3:00	The kicker: you'd never coded, and Claude Code built all of it.

Numbers to show: generation time, stats found, lines checked, lines cut, and cost per segment.

Backup: a finished segment and a screen recording, in case the live run stalls or the wifi drops.

First prompt for Claude Code

Before pasting: export this doc as Markdown and save it as BRIEF.md in your project folder. Put each bible in a characters folder inside it, one file per character. Have your API key ready, and switch Claude Code to plan mode (Shift+Tab).

```
I've never coded before, so explain what you're doing in plain language
as you go. I'm building "The Receipts": a webpage where you type an NBA
or NFL topic and AI generates a 3-to-5-minute debate between a moderator
and three hosts.

Before anything else, read BRIEF.md (the full project spec) and every
file in the characters folder (the character bibles). Follow them. If
something is unclear or contradicts itself, ask me instead of guessing.

TODAY'S GOAL: STEP 1, AN UGLY VERSION I CAN SEE WORKING

Build only this:
```

1. A Python program that takes a topic and uses the Claude API to write a debate script that follows the bibles and the Debate rules section of BRIEF.md. Use the current Sonnet model for the writing (look up the exact model name in Anthropic's docs).
2. The script follows the segment flow in BRIEF.md: a caller asks the question, the moderator takes it, the hosts argue, and the moderator wraps up. For now the moderator does not declare a winner; scoring comes later. Aim for 450 to 750 words (3 to 5 minutes at speaking pace) and stop when the arguments run out.
3. For now, hosts must not cite specific numbers or stats. Real, fact-checked stats come in step 2, so arguments use general claims.
4. Save each script as a JSON file in a runs folder, one folder per run named by date and time. Save the topic, the format the AI chose (debate, buy or sell, or prediction), and every line with: line number, speaker, text, delivery (calm, heated, laughing, and so on), and argument number.
5. A plain local webpage with a text box and a Go button. When I submit a topic, it shows a "generating" message, then plays the script as subtitles: one line at a time with the speaker's name, each line staying on screen about as long as it takes to say it (about 150 words per minute).
6. One command to start everything, and a README explaining how to run it.

NOT YET (later steps, don't build these now): the topic check, stat research, fact-checking, the judge and scoring, pixel art, and voices.

SETUP RULES

- Check whether Python is installed. If it isn't, walk me through installing it.
- Keep my API key in a .env file that is never shown, printed, or committed. Tell me exactly how to add my key to it.
- Set up git and commit after each piece works, so we can undo mistakes.
- Create a CLAUDE.md that tells future sessions to read BRIEF.md and the bibles, explains how to run the project, and says which step we're on.
- Keep the code simple and readable: Python for the server, plain HTML and JavaScript for the page, no frameworks.

HOW TO WORK WITH ME

First, show me a short plan for the whole project (all the steps in

```
BRIEF.md) and a detailed plan for step 1, then wait for my OK.

After building, run these three test topics yourself:
- "Is Patrick Mahomes the best quarterback of all time?"
- "Should the Lakers buy or sell before the trade deadline?"
- "Who wins the Super Bowl this season?"

Then show me where the scripts were saved, tell me how to open the page
and try my own topic, explain each file you created in one sentence, and
list what step 2 will add.
```

After step 1 works, read the three test scripts, fix the bibles wherever a host sounds generic, then ask for one pipeline step at a time, in the order under How it's built.